

Audion Media Tools - User Guide

Current for **Audion Media Tools** as of 2026-06-15.

This guide covers the user-facing project capabilities: workspace paths, yt-dlp downloads, audio workflows, archive and mezzanine codecs, storage and delivery encodes, remix, FPS conversion, LUT/grading, diagnostics, reports and the built-in terminal.

For the shorter project map, see **README.md** and **Docs\README.md**. For deeper Russian procedure notes, see **Docs\docs**.

What It Is

Audion Media Tools is a portable Windows media workbench built around FFmpeg, ffprobe, yt-dlp, Deno, 7-Zip and a NiceGUI shell.

It is not a generic FFmpeg launcher. The project is designed as a controlled production surface: profiles, dry-run command preview, reports, logs, path cache, LUT cache, hardware checks and tooltips on the controls that carry risk or ambiguity.

Quick Start

Start the GUI:

```
launcher_gui.cmd
```

If the portable environment is missing or damaged:

```
builder_main.cmd
```

Default local endpoint:

```
http://127.0.0.1:8080/
```

Manual debug launch on another port:

```
runtime\python.exe system_core\ui_nicegui\app.py --host 127.0.0.1 --port 8081 --no-browser
```

Workspace Model

The top workspace strip controls:

- **Source** - input folder.
- **OUT** / **Transcoded** - output folder.

Path fields are read-only selectors with history and pinned entries. Use the buttons beside them to choose folders. Cleanup buttons require confirmation.

Normal batch media operations scan Source recursively and mirror the Source subfolder structure under OUT. Outputs are written as **OUT\<operation>\<suffix>\<Source mirror>\<original name>.<ext>**. A custom OUT folder simply becomes the root of that same layout. This prevents files such as **Source\Day1\clip.mov** and **Source\Day2\clip.mov** from collapsing into one flat output folder.

Tooltips

The GUI has delayed tooltips over buttons, modes and parameters.

- show delay: **1000 ms**;
- hide delay: **100 ms**.

If a control is unclear, hover it for about a second. Tooltips are especially important for audio workflow modes, stream selection, LUFs, resampling, hardware encode/decode, remux pairs and cleanup actions.

Segmented buttons align edge tooltips to the left or right edge so long text does not get clipped.

How To Run An Operation

1. Pick **Source** and **OUT**.
2. Open an operation page on the left.
3. Choose a profile.

4. Change only the parameters you need.
5. Use `Command` for a dry-run preview if the operation is new.
6. Press the run button in the operation header.

Profiles provide a useful baseline. Fields below the profile let you override details. Checkboxes should not imply hidden actions unless a profile explicitly enables them.

Documentation Map

- `Docs\docs\AUDION_MEDIA_TOOLS_RU.md` - architecture, folders, GUI principles.
- `Docs\docs\WORKSPACE_REPORTS_TERMINAL_RU.md` - Source/OUT, path cache, terminal and reports.
- `Docs\docs\YOUTUBE_DOWNLOAD_RU.md` - yt-dlp profiles, batch mode, subtitles and cookies.
- `Docs\docs\AUDIO_WORKFLOWS_RU.md` - extract audio, update audio in video, convert audio files.
- `Docs\docs\ENCODING_PIPELINES_RU.md` - archive, editing codecs, storage and delivery encodes.
- `Docs\docs\REMUX_FPS_LUT_RU.md` - remux, FPS conversion, LUT and HDR-to-SDR.
- `Docs\TRIM_RU.md` - trimming in full: the working cycle, the three actions, precision, refusals.
- `Docs\MPV_HOTKEYS_RU.md` - the player's keys: the editing ones first, then all 197.
- `Docs\TRIM_MATRIX_RU.md` - what may be cut into what.
- `Docs\MEASUREMENTS_RU.md` - the measurement log: what was tested and what came out.
- `Docs\docs\HARDWARE_DIAGNOSTICS_RU.md` - diagnostics, hardware cache, Profile Doctor and test matrix.
- `Docs\docs\MEDIA_KNOWN_PITFALLS_RU.md` - common mistakes and fixes.
- `Docs\docs\MEDIA_SMOKE_TEST_CHECKLIST_RU.md` - post-change and pre-release checks.

Operation Pages

Diagnostics

Use this page first when the project does not see tools, profiles or hardware backends.

Main operations:

- `Inventory` - counts project folders.
- `Selftest` - checks FFmpeg, ffprobe, yt-dlp and Deno.

- **Profile Doctor** - compares manifest operations, **Scripts*.cmd** wrappers and **config\script_profiles.yaml**.
- **Hardware Capabilities** - checks CPU, CUDA/NVENC, QSV, AMF/D3D11VA (H.264, HEVC and AV1), SVT-AV1 and dav1d.
- **Preset Test Matrix** - runs presets on a short fixture and separates **MISS** from real failures.

Tool Installation

Explicit payload installation:

- 7-Zip;
- FFmpeg;
- yt-dlp.

For a full environment build, **builder_main.cmd** is usually more convenient. The GUI page is useful for targeted repairs.

Download / YouTube

Uses yt-dlp. Supports a single URL or batch mode via **Download\urls.txt**.

Main profiles:

- **Best MP4 video** - best general MP4/M4A merge.
- **1080p AVC MP4** - compatible H.264/MP4.
- **Safe Safari** - Safari client, IPv4 and retries for fragile links.
- **AV1 if available** - prefers AV1 video.
- **Audio MP3** - audio-only MP3.
- **Batch from urls.txt** - reads a link list.

Key parameters: resolution, video codec, audio format, container, source mode, retry/IPv4/metadata/no-playlist, subtitles, fragments, client, subtitle languages, cookies and download archive.

Audio Streams

The audio page has three segmented workflow buttons:

- **Extract audio** - create separate audio files from video containers.
- **Update video audio** - create a new video file with video copied and selected audio changed in container.

- **Convert audio** - process standalone WAV/FLAC/M4A/MP3/AAC/Opus/MKA/OGG/APE/WV and other audio files from Source when the local FFmpeg can decode them.

Capabilities:

- Mode buttons: Copy/remux, lossless/PCM, lossy delivery and analysis; they update the form parameters.
- Copy, WAV, FLAC, ALAC, M4A/AAC, MP3, Opus.
- Copy container policy: Auto, M4A, MKA or native codec file; Auto extracts stereo/mono AAC as M4A, AC3/E-AC3/DTS as native codec files, and complex multichannel cases as MKA.
- Copy/source, 44.1/88.2/176.4 kHz and 48/96/192 kHz.
- Copy/source, 16-bit, 24-bit, 32-bit float.
- SoX/libsoxr 100% quality (**precision=33**) for sample-rate changes.
- First stream, all streams, stream by index or language tag.
- LUFS off/report/one-pass/two-pass.
- Keep, stereo, mono, dual mono and 5.1 Matrix.
- LAME V0, Insane 320 or manual bitrate. The same scale is available on the encode pages when **Audio = MP3**; manual bitrate stops at 320 kbps, the MPEG-1 Layer III ceiling.

Audio operations also write **audio_report.md/json**.

Archive

Long-term master formats:

- FFV1 MKV + FLAC;
- x264 lossless MKV;
- dual archive.

Use Archive when technical preservation matters more than output size.

Editing Codecs

Mezzanine formats for NLE workflows:

- ProRes Proxy / LT / 422 / HQ;
- DNxHR HQ / HQX;
- MOV or MXF.

This is the right page for preparing files for editing, not for final web delivery.

Storage

Compact high-quality files for a library or transfer:

- x264 / H.264;
- x265 / HEVC;
- SVT-AV1.

Key parameters: encode backend, decode backend, audio, bitrate, container, preset, resolution, AV1 preset, pixel format, CRF and tuning.

Delivery

Review, upload and publishing profiles:

- Review H.264;
- Upload H.264;
- Upload HEVC 10-bit;
- SVT-AV1 delivery.

Hardware encoding is selected separately in the **Encoder** block: CPU, NVENC, AMF or QSV. CPU shows **CPU preset** and **CRF**; hardware encoders show their native preset/quality controls and **CQ/QP**.

Remux

Changes containers or stream layout without re-encoding where possible. Source is scanned recursively, and each file format is detected automatically.

Pipeline modes:

- **Video** keeps video streams only.
- **Audio** extracts/copies audio streams only.
- **Video&Audio** copies video and audio together.

Video and **Video&Audio** use MP4, MOV, MKV or MXF output containers; a file whose codec the target container cannot hold is skipped with the reason in the log. **Audio** uses M4A, ALAC/M4A, MKA, AAC, AC3, EAC3, DTS, FLAC, WAV, MP3, OPUS or OGG targets.

The GUI does not ask for a manual source format. The backend checks each file and skips incompatible cases with a log reason. Same-container copy is allowed when the stream layout changes, for example MP4 without audio.

Which way up. A row of buttons in the same section: **as shot**, **90 clockwise**, **90 counter-clockwise**, **upside down**, **mirror left-right**, **mirror top-bottom**.

It answers a familiar complaint: the camera was turned on its side for a vertical shot, wrote an ordinary landscape file and said nothing about it - and every clip then has to be turned by hand in the NLE. Here the turn is written into the container, and the editor opens the clip upright.

No pixel is touched. Measured on a 960x540 take: the file was 6 085 307 bytes and came out 6 085 304 - the difference is in the header, the frame hashes match exactly, and a decoder returns 540x960.

The limits, measured rather than assumed:

- **MP4, MOV and MKV** store it. **MXF does not** - it accepts the option and silently writes an unturned file, so the section refuses before the run;
- it is offered for camera **H.264 and HEVC**; ProRes and DNxHR are skipped with the reason;
- it is an instruction to whatever displays the file. Resolve, Premiere and mpv follow it; a simple viewer may not.

Trim

Cuts the head, the tail or the middle out of a camera file **without re-encoding**: the streams are copied, no generation is lost, and metadata and timecode travel into the result.

What to choose. Three buttons, all three about the piece between the points.

Keep it - what lies between IN and OUT stays and the rest goes. Which end goes is already in the fields, and saying it twice would only let the two answers disagree: IN alone drops the head (the slate, the walk-up, the false start), OUT alone drops the tail (the walk-away and the camera stop), both drop both ends.

Cut it out - the opposite: the piece between the points disappears and the two ends are joined into one file. The fluffed line, the phone ringing, the assistant walking through shot. Nothing is re-encoded here either: both pieces are copied and glued packet to packet.

Split in two - one point, nothing discarded, two files out.

In the cut-out mode one rule runs the other way from every other cut. Everywhere else the head of a piece lands on the nearest keyframe even when it sits earlier, because extra frames cost nothing. Here they cost the wrong thing - part of what was being removed would survive. So the tail starts at the **first keyframe at or after OUT**, and the log says how much more than asked for went. When a keyframe sits exactly on the point, nothing extra goes at all.

The sound is cut on the sample at the joint: PCM is rewritten in the same format and the piece is given its own length, or the sound stops with the last frame. Measured on a 25 fps take: without this the joint drifted by 72 ms, then by 16 ms; it now lands on exactly 1 200 000 samples across 25.000 s.

How the points are set. Two ways, both exact.

In the player. `Open in the player` shows the chosen take in mpv. There, space plays and pauses, `,` and `.` step exactly one frame back and forward, `b` plays backwards, and `l` sets the A point - press it again for B, and the piece between them loops, so the cut is heard before it is made. `Take A and B` then carries both points into the fields.

With hands on the panel instead, `Mark IN here` and `Mark OUT here` take the position the player stands at, write it into the field and move the loop point there, so the piece keeps looping.

The player is its own program: it can live on a second monitor, and if it is closed the panel carries on. It is installed once, with `install\Install-Portable-mpv.cmd`.

By typing. A timecode: `00:01:28,400`, or plain seconds. It needs nothing but the number, and it stays available whether or not the player is installed.

What the section promises. The head of the kept piece lands on the nearest keyframe before the point asked for - a copied stream cannot begin anywhere else

- and the log says where it actually went. The tail is counted in frames from the exact rate, so it lands on the frame asked for. The rate is taken from the file as it is: `ffprobe` returns an exact fraction and nothing is inferred - cameras shoot 23.976 and an honest 24.000 alike.

When the section refuses - before the run, not after. MP4 will not take ProRes, MXF will not take compressed audio, a request for an audio track that does not exist is rejected, and camera RAW inside MXF (Sony X-OCN, ARRIRAW) has no FFmpeg decoder at all. Each case is stated in words, with the alternative that does work.

The same from the command line: `Scripts\ff-trim-start.cmd`, `ff-trim-end.cmd`, `ff-trim-both.cmd`, `ff-trim-middle.cmd`. The arithmetic is shared with the panel: a file cut by the wrapper and one cut by the panel came out byte for byte identical.

Details, compatibility tables and measurements: `Docs\TRIM_MATRIX_RU.md` and `Docs\MEASUREMENTS_RU.md`.

FPS

The source FPS is read from the video stream. The user chooses:

- **Varispeed** or **Conform**;
- a target from the full range: **16**, **18**, **23.976**, **24**, **25**, **29.97**, **30**, **48**, **50**, **59.94**, **60**. Fractional rates are kept as exact fractions, and the source rate may be anything at all - 8000 fps was tested;
- output profile: H.264 MP4, H.265 MP4, ProRes MOV/MXF, DNxHR MOV/MXF.
- for ProRes/DNxHR in Apple MOV: PCM 16/24/32-float with the selected work rate preserved up to 176.4/192 kHz;
- for ProRes/DNxHR in MXF: PCM 16/24 with a final 48 kHz output; **2x/4x** remains an internal processing rate;
- the CLI FPS wizard exposes the same profiles, depths and **x1/x2/x4**, and both front-ends use the same command contract.

Color / LUT

The LUT page covers:

- LUT picker/cache from **LUTs**;
- **LUT x264**, **LUT HEVC Main10**, **LUT ProRes 422**;
- **Pre-expose + LUT**;
- **Pregamma + LUT**;
- **HDR to SDR HabLe**;
- targets: x264, x265, SVT-AV1, ProRes 422, ProRes LT.

Pregamma/pre-expose modes show an 18% gray preview that makes the direction of the pre-LUT gamma change visible.

Hardware Stack

Supported choices:

- CUDA / NVENC;
- Intel QuickSync / QSV;
- AMD AMF / D3D11VA;
- dav1d for AV1 decode;

- SVT-AV1 for AV1 encode;
- CPU fallback.

Availability depends on the FFmpeg build, drivers and actual hardware. Hardware backends are checked against the latest `Hardware Capabilities` cache.

Terminal, Progress And Reports

The right side contains:

- operation status;
- global progress bar;
- terminal log;
- `Command` dry-run preview;
- `Logs`;
- `Report`;
- `CONFIG`;
- expanded terminal view;
- manual command line with Shell, CWD, history and pins.

Each media operation writes `batch_report.md` and `batch_report.json` under `report\...`. Audio operations also write `audio_report.md/json`.

Safe Working Rules

- Use `Command` before a new real run.
- Use `Test on first file` for large folders.
- Avoid `Overwrite` unless OUT is intentionally disposable.
- Run `Hardware Capabilities` before hardware encodes.
- Check the selected `.cube` before LUT work.
- Check `Download\urls.txt` before YouTube batch mode.
- Read cleanup confirmations carefully.
- Do not run two operations in one GUI instance at the same time.

Verification

```
runtime\python.exe -m py_compile system_core\ui_nicegui\app.py
system_core\services\media_service.py Scripts\Common\script_runner.py
runtime\python.exe system_core\ui_nicegui\app.py --smoke
```

For mirrored OUT validation, see [Docs\docs\MEDIA_SMOKE_TEST_CHECKLIST_RU.md](#).

GUI Manifest Review

The current [config\tool_manifest.yaml](#) is the structured source for pages, bilingual labels, field types, defaults, conditional visibility, tooltips, presets, and command bindings. This guide supplies the human interpretation: which workflow fits an intent, which combinations are unsafe, and what evidence proves success.

Before release, compare manifest options with the rendered GUI, dry-run command, backend validation, and report. A new field is incomplete until its purpose and acceptance consequence are understandable here. A removed field must not survive as a promise in documentation.